ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ИМЕНИ ПАТРИСА ЛУМУМБЫ

Факультет искусственного интеллекта

Отчет

о выполнении лабораторной работы № 4

“Глубокие сети прямого распространения”

по дисциплине "Нейронные сети"

Вариант задания № 13

Выполнил студент группы ЗФИбд-01-24

Князев А. М._____

Проверил и принял

доцент кафедры прикладного ИИ Леонов С.С._____

с оценкой _____

Москва, 2026

Цель работы: изучение глубоких полносвязных нейронных сетей прямого распространения и их применение в задачах бинарной классификации, многоклассовой классификации и регрессии.

Основные этапы работы:

1. Построить глубокую полносвязную сеть прямого распространения для бинарной классификации эмоциональной окраски отзывов из набора данных IMDB. Закодировать отзывы мульти-горячими векторами, обучить сеть заданной вариантом структуры и сопоставить её качество с базовым классификатором по доле верных ответов на тестовом подмножестве.
2. Построить полносвязную сеть для многоклассовой классификации новостных лент из набора данных Reuters (46 классов). Обучить сеть с выходным слоем softmax и функцией потерь — категориальной перекрёстной энтропией, сравнить результаты с базовой моделью и проанализировать влияние заданных вариантом гиперпараметров.
3. Построить регрессионную сеть для прогноза медианной стоимости жилья (Boston Housing). Ввиду малого объёма данных оценить качество методом перекрёстной проверки по K блокам, вычислить среднюю абсолютную погрешность (MAE) по блокам и сопоставить её с базовой моделью.

Программное обеспечение: Python 3.x, Jupyter Notebook, библиотеки PyTorch, NumPy и Matplotlib.

Сценарий выполнения работы

Этап 1. Бинарная классификация отзывов IMDB

Набор данных IMDB содержит текстовые рецензии на фильмы с двумя метками тональности — положительная и отрицательная. Из локального архива берётся по 700 отзывов каждой тональности из обучающей и тестовой частей архива — всего 2800 отзывов. Каждый отзыв преобразуется в вектор размерности 5000 по схеме мульти-горячего кодирования: компонента равна 1, если соответствующее слово из словаря 5000 наиболее частых слов встречается в отзыве, и 0 иначе. Данные разделены на обучающее, контрольное и тестовое подмножества

(1792 / 448 / 560 примеров).

Процесс обучения (общий для всех этапов). Обучение ведётся мини-батчами: обучающее подмножество перемешивается и разбивается на небольшие порции примеров, которые подаются в сеть по очереди. Для каждой порции выполняется прямой проход — вычисление выхода сети, после чего функция потерь сравнивает выход с целевыми значениями. Методом обратного распространения ошибки вычисляются градиенты функции потерь по всем весам сети, и оптимизатор Adam обновляет веса, подбирая для каждого параметра свой адаптивный шаг на основе скользящих оценок первого и второго моментов градиента (скорость обучения $1 \cdot 10^{-3}$ в задачах классификации и $5 \cdot 10^{-4}$ в задаче регрессии). Полный проход по обучающему подмножеству составляет одну эпоху; обучение длится заданное число эпох. После каждой эпохи качество измеряется на контрольном подмножестве, что позволяет следить за обобщающей способностью сети и вовремя замечать переобучение. Метрика качества для классификации — доля верных ответов (accuracy), для регрессии — средняя абсолютная погрешность (MAE).

Согласно варианту 13 сеть содержит три скрытых слоя по 16, 32 и 16 нейронов с функциями активации relu, relu и tanh; выходной слой — один нейрон с сигмоидой. В качестве функции потерь по варианту используется среднеквадратическая погрешность (MSE), оптимизатор — Adam. Результаты сопоставляются с базовым классификатором (2 слоя по 16 нейронов, relu-relu, бинарная перекрёстная энтропия).

Таблица 1 — Структура сети (этап 1, вариант 13)

Параметр	**Значение**
Размер входного вектора	5000
Скрытые слои	16 → 32 → 16
Функции активации	relu, relu, tanh
Выходной слой	1 нейрон, sigmoid
Функция потерь	MSE (среднеквадратическая погрешность)
Оптимизатор	Adam

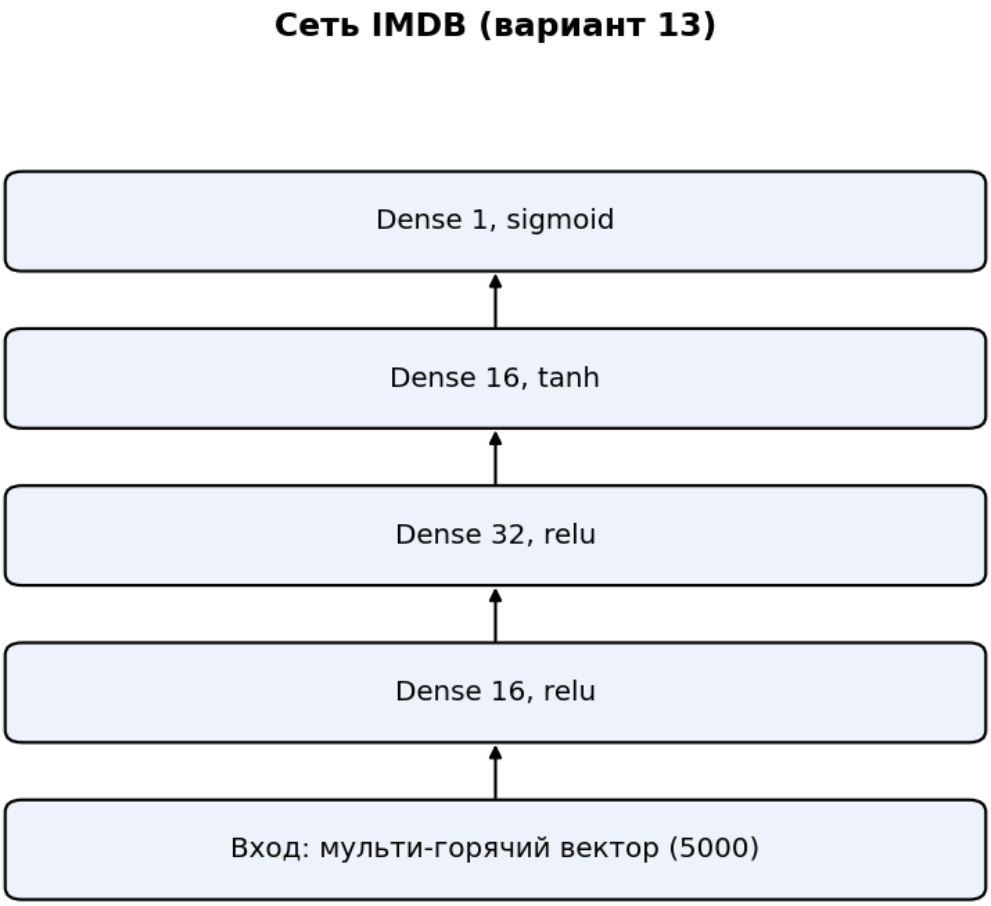


Рис. 1 — Схема сети IMDB (вариант 13)

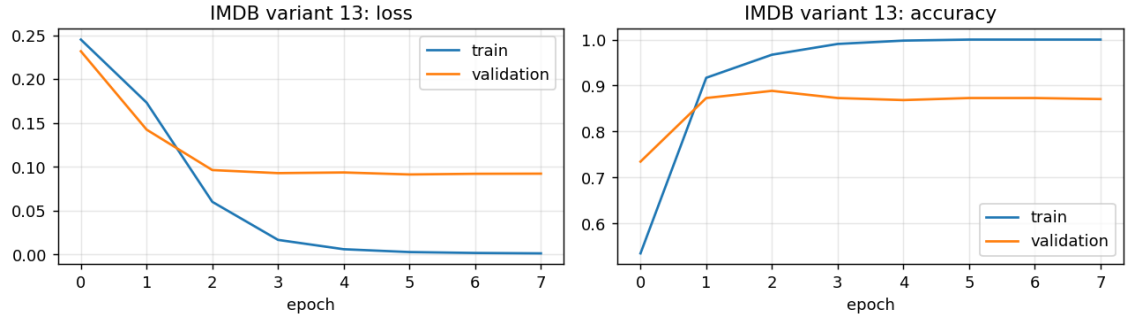


Рис. 2 — Графики функции потерь и точности (IMDB, вариант 13)

Таблица 2 — Результаты на тестовом подмножестве (этап 1)

Модель	**Test loss**	**Test accuracy**
Вариант 13 (16-32-16, MSE)	0.0769	0.8964
Базовая (16-16, BCE)	0.2458	0.8946

Обе модели достигают сопоставимой точности около 0.89, причём

вариант 13 (0.8964) опережает базовую модель (0.8946) лишь на ~0.002 — разница в пределах статистического шума. По графикам видно, что точность на контрольном подмножестве выходит на насыщение уже после двух-трёх эпох, тогда как на обучающем продолжает расти: дальнейшее обучение ведёт не к улучшению обобщения, а к переобучению. Меньшее значение функции потерь у варианта (0.0769 против 0.2458) объясняется не качеством модели, а разными функциями потерь (MSE и бинарная перекрёстная энтропия) и поэтому напрямую несравнимо — для сравнения моделей корректна именно точность.

Этап 2. Многоклассовая классификация Reuters

Набор данных Reuters содержит короткие новостные ленты одноимённого агентства, отнесённые к 46 тематическим рубрикам. Каждый текст представлен последовательностью индексов слов, которая, как и в случае IMDB, переводится в мульти-горячий вектор по словарю из 10000 наиболее частых слов: компонента вектора равна 1, если соответствующее слово встречается в тексте, и 0 иначе. Метка кодируется индексом класса (от 0 до 45). Классы распределены неравномерно — отдельные рубрики представлены заметно чаще остальных, что усложняет задачу классификации. Данные разделены на обучающее, контрольное и тестовое подмножества (7185 / 1797 / 2246 примеров).

Согласно варианту 13 сеть содержит три скрытых слоя по 32, 64 и 64 нейрона с функциями активации tanh, mish и relu; выходной слой — 46 нейронов с softmax. Функция потерь — многокатегориальная перекрёстная энтропия, оптимизатор — Adam. Базовая модель — два слоя по 64 нейрона (relu-relu).

Таблица 3 — Структура сети (этап 2, вариант 13)

Параметр	**Значение**
Число классов	46
Скрытые слои	32 → 64 → 64
Функции активации	tanh, mish, relu
Выходной слой	46 нейронов, softmax
Функция потерь	категориальная перекрёстная энтропия

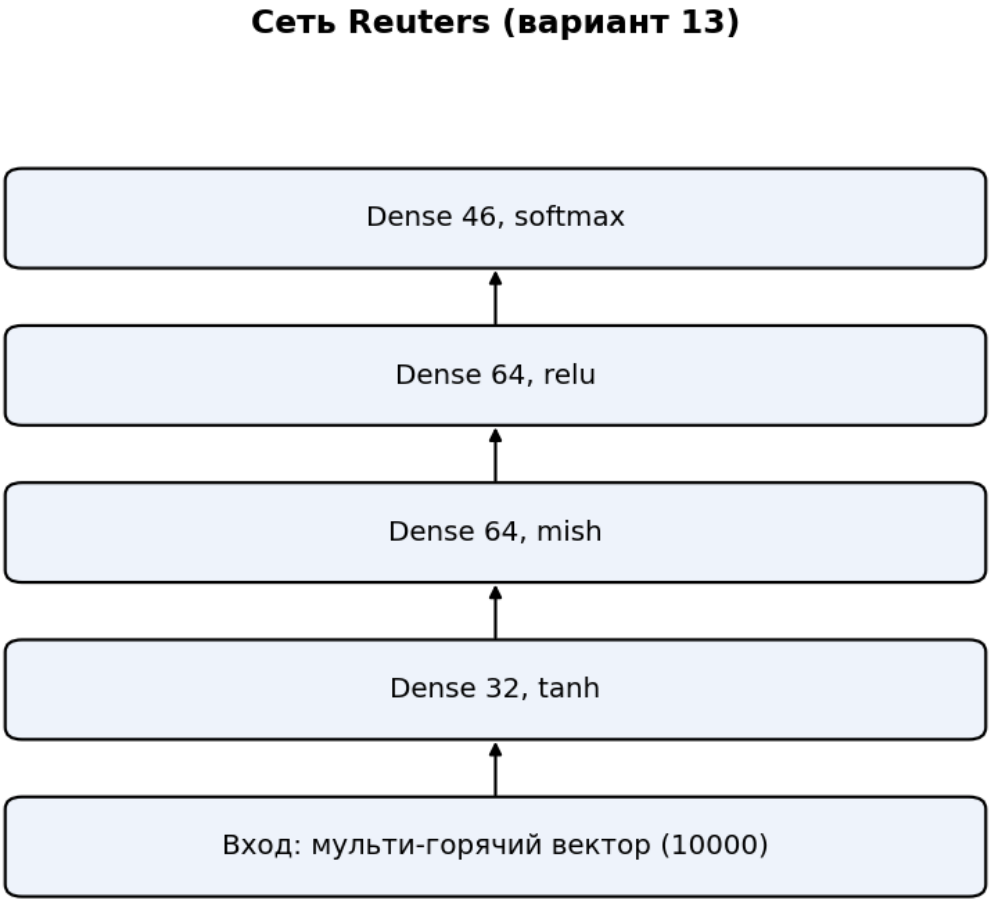


Рис. 3 — Схема сети Reuters (вариант 13)

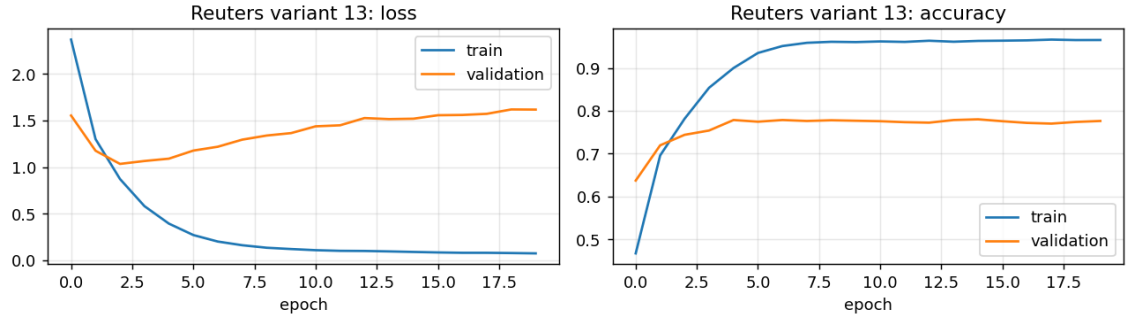


Рис. 4 — Графики функции потерь и точности (Reuters, вариант 13)

Таблица 4 — Результаты на тестовом подмножестве (этап 2)

Модель	**Test loss**	**Test accuracy**
Вариант 13 (32-64-64)	1.7665	0.7467
Базовая (64-64)	1.2476	0.7796

Базовая конфигурация (0.7796) показала точность выше варианта 13 (0.7467) примерно на 3 процентных пункта. Ключевая причина — узкое

«горлышко» в сети варианта: его первый скрытый слой содержит всего 32 нейрона, что меньше числа классов (46), поэтому информация о 46 рубриках сжимается в 32-мерное представление ещё до расширяющего слоя, и часть различающих признаков теряется. У базовой сети первый слой шире ($64 \geq 46$), и такого бутылочного горла нет. Нестандартное сочетание активаций (tanh, mish, relu) этот недостаток не компенсирует. Таким образом, заданные вариантом гиперпараметры для данной задачи не оптимальны.

Этап 3. Регрессия Boston Housing с перекрёстной проверкой по K блокам

Набор данных Boston Housing содержит 506 примеров с 13 числовыми признаками района Бостона (уровень преступности, среднее число комнат, доля старого жилья, удалённость от центров занятости и т. д.); целевая величина — медианная стоимость жилья в тысячах долларов. Признаки имеют разные масштабы, поэтому стандартизируются (приводятся к нулевому среднему и единичному стандартному отклонению) по статистикам обучающего блока.

Перекрёстная проверка по K блокам. Поскольку набор небольшой, качество оценивается методом K-блочной кросс-валидации. Данные делятся на K = 4 равных блока (число скрытых слоёв плюс один). Процедура повторяется K раз: на каждой итерации один блок выступает контрольным, а оставшиеся K−1 блоков — обучающими; на них сеть обучается заново, после чего на отложенном блоке считается MAE. Итоговая оценка — среднее MAE по всем четырём блокам; такой подход даёт более устойчивую оценку обобщающей способности, чем единичное разбиение.

Согласно варианту 13 сеть содержит три скрытых слоя по 32, 64 и 64 нейрона с функциями активации tanh, mish и relu; выходной слой — один нейрон без активации. Функция потерь — среднеквадратическая погрешность, метрика качества — средняя абсолютная погрешность (MAE), оптимизатор — Adam. Базовая модель — два слоя по 64 нейрона (relu-relu).

Таблица 5 — Структура сети (этап 3, вариант 13)

Параметр	**Значение**
Число признаков	13
Скрытые слои	32 → 64 → 64
Функции активации	tanh, mish, relu
Выходной слой	1 нейрон, линейный
Функция потерь / метрика	MSE / MAE
Число блоков K	4

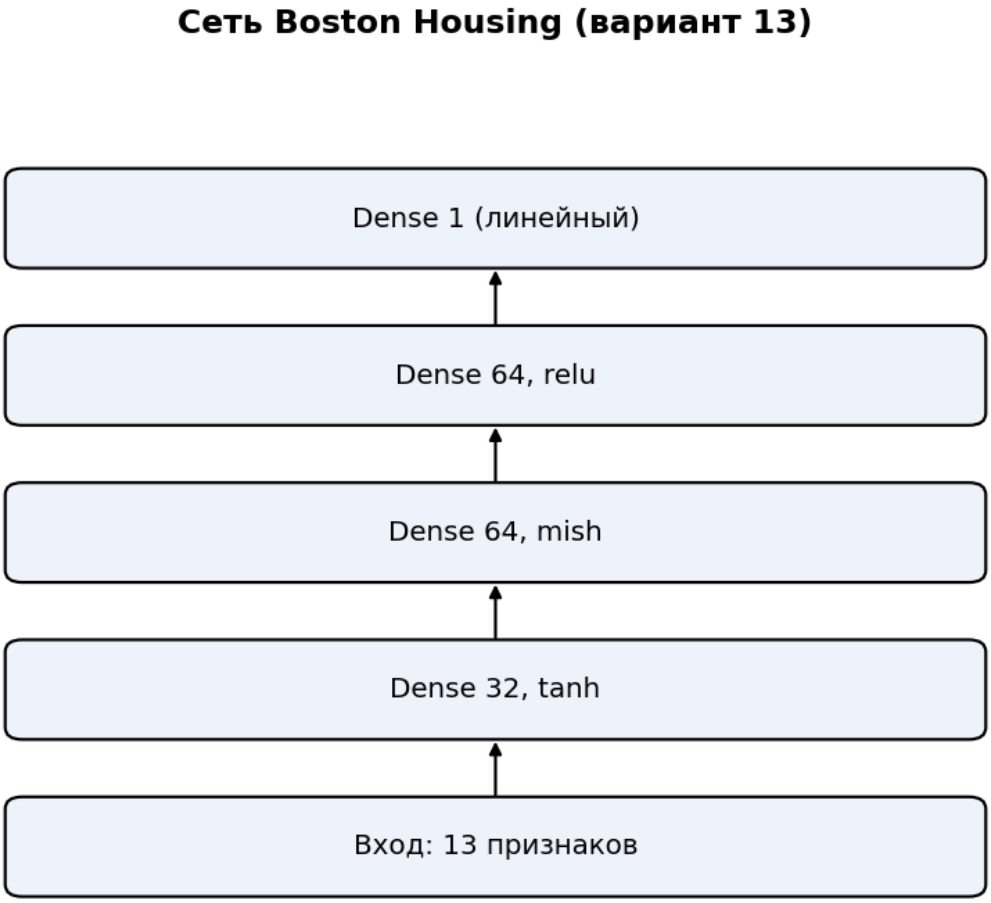


Рис. 5 — Схема сети Boston Housing (вариант 13)

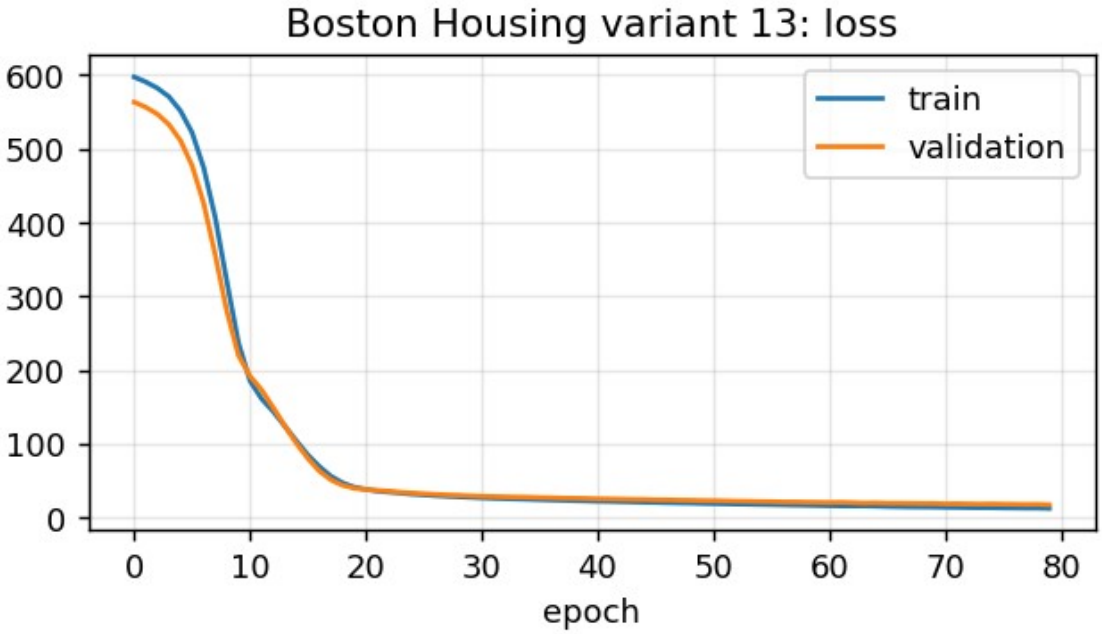


Рис. 6 — График функции потерь при обучении (Boston Housing, вариант 13)

Таблица 6 — Результаты перекрёстной проверки (MAE по блокам)

Модель	**MAE по блокам**	**Средняя MAE**
Вариант 13 (32-64-64)	[2.82, 2.64, 3.28, 3.01]	2.94
Базовая (64-64)	[2.89, 2.75, 2.82, 2.62]	2.77

Средняя абсолютная погрешность прогноза составляет около 2.9 тыс. долларов; при медианной стоимости жилья порядка 20 тыс. долларов это соответствует относительной ошибке примерно 13–14% — приемлемый результат для столь малого набора. Разброс MAE по блокам (от 2.64 до 3.28 у варианта 13) показывает, насколько оценка чувствительна к конкретному разбиению данных; именно поэтому применяется усреднение по K блокам, дающее более устойчивую оценку, чем единичное разбиение. Значения варианта 13 (2.94) и базовой модели (2.77) близки — на этой задаче выбор архитектуры в заданных пределах слабо влияет на точность.

Код программы

```
import random
import zipfile
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import torch
import torch.nn as nn
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import KFold, train_test_split
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset

DEVICE = torch.device(
    "cuda" if torch.cuda.is_available()
    else "mps" if torch.backends.mps.is_available()
    else "cpu"
)
SEED = 13
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)


def existing_path(*variants: str) -> Path:
    for variant in variants:
        path = Path(variant)
        if path.exists():
            return path
    return Path(variants[0])


def local_dataset_file(filename: str) -> Path:
    path = existing_path(f"data/{filename}", f"../data/{filename}")
    if not path.exists():
        raise FileNotFoundError(
            f"Не найден {filename}. Скачай его один раз в папку data рядом
с ноутбуком."
        )
    return path


print(f"Device: {DEVICE}")

def activation(name: str) -> nn.Module:
    activations = {
        "relu": nn.ReLU(),
        "tanh": nn.Tanh(),
```

```
        "mish": nn.Mish(),
        "elu": nn.ELU(),
        "leaky_relu": nn.LeakyReLU(0.1),
    }
    return activations[name]


def make_mlp(input_size: int, hidden: list[int], activations: list[str],
output_size: int) -> nn.Sequential:
    layers = []
    prev = input_size
    for width, act in zip(hidden, activations):
        layers.append(nn.Linear(prev, width))
        layers.append(activation(act))
        prev = width
    layers.append(nn.Linear(prev, output_size))
    return nn.Sequential(*layers)


def make_loader(x: np.ndarray, y: np.ndarray, batch_size: int = 64,
shuffle: bool = True) -> DataLoader:
    dataset = TensorDataset(torch.tensor(x, dtype=torch.float32),
torch.tensor(y))
    return DataLoader(dataset, batch_size=batch_size, shuffle=shuffle)

def count_correct(logits: torch.Tensor, y: torch.Tensor, task: str) ->
int:
    if task == "regression":
        return 0
    if task == "multiclass":
        return int((logits.argmax(dim=1) == y).sum().item())
    if task == "binary_probability":
        pred = (logits.view(-1) >= 0.5).float()
    else:
        pred = (torch.sigmoid(logits).view(-1) >= 0.5).float()
    return int((pred == y.view(-1)).sum().item())


def evaluate(model: nn.Module, loader: DataLoader, criterion: nn.Module,
task: str) -> tuple[float, float]:
    model.eval()
    total_loss = 0.0
    correct = 0
    total = 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            out = model(x)
            loss = criterion(out, y)
            total_loss += float(loss.item()) * len(x)
```

```
            correct += count_correct(out, y, task)
            total += len(x)
    accuracy = correct / total if task != "regression" else 0.0
    return total_loss / total, accuracy


def train_model(
    model: nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    criterion: nn.Module,
    task: str,
    epochs: int = 10,
    lr: float = 1e-3,
) -> dict[str, list[float]]:
    model.to(DEVICE)
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)
    history = {"loss": [], "val_loss": [], "acc": [], "val_acc": []}

    for _ in range(epochs):
        model.train()
        train_loss = 0.0
        train_correct = 0
        train_total = 0
        for x, y in train_loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            optimizer.zero_grad()
            out = model(x)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()
            train_loss += float(loss.item()) * len(x)
            train_correct += count_correct(out, y, task)
            train_total += len(x)

        val_loss, val_acc = evaluate(model, val_loader, criterion, task)
        history["loss"].append(train_loss / train_total)
        history["acc"].append(train_correct / train_total if task !=
"regression" else 0.0)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

    return history

def plot_history(history: dict[str, list[float]], title: str,
show_accuracy: bool = True) -> None:
    cols = 2 if show_accuracy else 1
    fig, axes = plt.subplots(1, cols, figsize=(10 if show_accuracy else 5,
3))
    if not show_accuracy:
        axes = [axes]
```

```
    axes[0].plot(history["loss"], label="train")
    axes[0].plot(history["val_loss"], label="validation")
    axes[0].set_title(f"{title}: loss")
    axes[0].set_xlabel("epoch")
    axes[0].grid(True, alpha=0.3)
    axes[0].legend()

    if show_accuracy:
        axes[1].plot(history["acc"], label="train")
        axes[1].plot(history["val_acc"], label="validation")
        axes[1].set_title(f"{title}: accuracy")
        axes[1].set_xlabel("epoch")
        axes[1].grid(True, alpha=0.3)
        axes[1].legend()

    plt.tight_layout()
    plt.show()

def read_imdb(zip_path: Path, max_docs_per_class: int = 700) ->
tuple[list[str], np.ndarray]:
    if not zip_path.exists():
        raise FileNotFoundError(f"Не найден архив IMDB: {zip_path}")

    texts = []
    labels = []
    with zipfile.ZipFile(zip_path) as archive:
        for split in ["train", "test"]:
            for label_name, label in [("neg", 0), ("pos", 1)]:
                prefix = f"aclImdb/{split}/{label_name}/"
                names = [n for n in archive.namelist() if
n.startswith(prefix) and n.endswith(".txt")]
                names = [n for n in names if "/._" not in n]
                for name in names[:max_docs_per_class]:
                    texts.append(archive.read(name).decode("utf-8",
errors="ignore"))
                    labels.append(label)

    return texts, np.array(labels, dtype=np.float32)

texts, y = read_imdb(existing_path("IMDB/aclImdb.zip",
"../IMDB/aclImdb.zip"), max_docs_per_class=700)

x_train_text, x_test_text, y_train, y_test = train_test_split(
    texts, y, test_size=0.2, stratify=y, random_state=SEED
)
x_train_text, x_val_text, y_train, y_val = train_test_split(
    x_train_text, y_train, test_size=0.2, stratify=y_train,
random_state=SEED
)
```

```
vectorizer = CountVectorizer(max_features=5000, binary=True,
stop_words="english")
x_train =
vectorizer.fit_transform(x_train_text).astype("float32").toarray()
x_val = vectorizer.transform(x_val_text).astype("float32").toarray()
x_test = vectorizer.transform(x_test_text).astype("float32").toarray()

print(x_train.shape, x_val.shape, x_test.shape)

def column(values: np.ndarray) -> np.ndarray:
    return values.reshape(-1, 1).astype("float32")

imdb_model = nn.Sequential(
    make_mlp(x_train.shape[1], [16, 32, 16], ["relu", "relu", "tanh"], 1),
    nn.Sigmoid(),
)
imdb_baseline = make_mlp(x_train.shape[1], [16, 16], ["relu", "relu"], 1)

train_loader = make_loader(x_train, column(y_train), batch_size=64)
val_loader = make_loader(x_val, column(y_val), batch_size=128,
shuffle=False)
test_loader = make_loader(x_test, column(y_test), batch_size=128,
shuffle=False)

imdb_history = train_model(
    imdb_model, train_loader, val_loader, nn.MSELoss(),
task="binary_probability", epochs=8
)
baseline_history = train_model(
    imdb_baseline, train_loader, val_loader, nn.BCEWithLogitsLoss(),
task="binary", epochs=8
)

imdb_loss, imdb_acc = evaluate(imdb_model, test_loader, nn.MSELoss(),
"binary_probability")
base_loss, base_acc = evaluate(imdb_baseline, test_loader,
nn.BCEWithLogitsLoss(), "binary")

print(f"IMDB variant: test_loss={imdb_loss:.4f}, test_acc={imdb_acc:.4f}")
print(f"IMDB baseline: test_loss={base_loss:.4f},
test_acc={base_acc:.4f}")

plot_history(imdb_history, "IMDB variant 13")
plot_history(baseline_history, "IMDB baseline")

reuters_path = local_dataset_file("reuters.npz")

with np.load(reuters_path, allow_pickle=True) as data:
    reuters_sequences = data["x"]
```

```
    reuters_labels = data["y"].astype("int64")

# Повторяем стандартную схему keras.datasets.reuters.load_data:
перемешивание с seed=113
# и разделение 80/20 на train/test.
indices = np.arange(len(reuters_sequences))
rng = np.random.default_rng(113)
rng.shuffle(indices)
reuters_sequences = reuters_sequences[indices]
reuters_labels = reuters_labels[indices]

split = int(len(reuters_sequences) * 0.8)
x_train_seq = reuters_sequences[:split]
y_train_full = reuters_labels[:split]
x_test_seq = reuters_sequences[split:]
y_test = reuters_labels[split:]

NUM_WORDS = 10000


def vectorize_sequences(sequences, dimension: int = NUM_WORDS) ->
np.ndarray:
    result = np.zeros((len(sequences), dimension), dtype="float32")
    for row, sequence in enumerate(sequences):
        indexes = [index for index in sequence if index < dimension]
        result[row, indexes] = 1.0
    return result


x_train_full = vectorize_sequences(x_train_seq)
x_test = vectorize_sequences(x_test_seq)

x_train, x_val, y_train, y_val = train_test_split(
    x_train_full, y_train_full, test_size=0.2, stratify=y_train_full,
random_state=SEED
)

num_classes = int(max(y_train_full.max(), y_test.max()) + 1)
multi_model = make_mlp(x_train.shape[1], [32, 64, 64], ["tanh", "mish",
"relu"], num_classes)
multi_baseline = make_mlp(x_train.shape[1], [64, 64], ["relu", "relu"],
num_classes)
criterion = nn.CrossEntropyLoss()

multi_history = train_model(
    multi_model,
    make_loader(x_train, y_train, batch_size=64),
    make_loader(x_val, y_val, batch_size=128, shuffle=False),
    criterion,
    task="multiclass",
```

```
    epochs=20,
)
train_model(
    multi_baseline,
    make_loader(x_train, y_train, batch_size=64),
    make_loader(x_val, y_val, batch_size=128, shuffle=False),
    criterion,
    task="multiclass",
    epochs=20,
)

multi_loss, multi_acc = evaluate(multi_model, make_loader(x_test, y_test,
batch_size=128, shuffle=False), criterion, "multiclass")
base_loss, base_acc = evaluate(multi_baseline, make_loader(x_test, y_test,
batch_size=128, shuffle=False), criterion, "multiclass")

print(f"Reuters train/val/test:
{len(x_train)}/{len(x_val)}/{len(x_test)}")
print(f"Reuters classes: {num_classes}")
print(f"Multiclass variant: test_loss={multi_loss:.4f},
test_acc={multi_acc:.4f}")
print(f"Multiclass baseline: test_loss={base_loss:.4f},
test_acc={base_acc:.4f}")

plot_history(multi_history, "Reuters variant 13")

boston_path = local_dataset_file("boston_housing.npz")

with np.load(boston_path) as data:
    x = data["x"].astype("float32")
    y = data["y"].astype("float32").reshape(-1, 1)

x = StandardScaler().fit_transform(x).astype("float32")

kfold = KFold(n_splits=4, shuffle=True, random_state=SEED)
fold_mae = []
baseline_mae = []
first_fold_history = None

for fold, (train_idx, val_idx) in enumerate(kfold.split(x), start=1):
    reg_model = make_mlp(x.shape[1], [32, 64, 64], ["tanh", "mish",
"relu"], 1)
    reg_baseline = make_mlp(x.shape[1], [64, 64], ["relu", "relu"], 1)
    criterion = nn.MSELoss()

    train_loader = make_loader(x[train_idx], y[train_idx], batch_size=32)
    val_loader = make_loader(x[val_idx], y[val_idx], batch_size=32,
shuffle=False)

    history = train_model(reg_model, train_loader, val_loader, criterion,
```

```
task="regression", epochs=80, lr=5e-4)
    train_model(reg_baseline, train_loader, val_loader, criterion,
task="regression", epochs=80, lr=5e-4)

    reg_model.eval()
    reg_baseline.eval()
    with torch.no_grad():
        val_tensor = torch.tensor(x[val_idx], dtype=torch.float32,
device=DEVICE)
        pred = reg_model(val_tensor).cpu().numpy()
        base_pred = reg_baseline(val_tensor).cpu().numpy()

    fold_mae.append(float(np.mean(np.abs(pred - y[val_idx]))))
    baseline_mae.append(float(np.mean(np.abs(base_pred - y[val_idx]))))

    if first_fold_history is None:
        first_fold_history = history

print(f"Boston Housing samples/features: {x.shape[0]}/{x.shape[1]}")
print(f"Regression KFold MAE: {np.round(fold_mae, 2).tolist()},
mean={np.mean(fold_mae):.2f}")
print(f"Regression baseline MAE: {np.round(baseline_mae, 2).tolist()},
mean={np.mean(baseline_mae):.2f}")

plot_history(first_fold_history, "Boston Housing variant 13",
show_accuracy=False)
```

Вывод

В ходе работы построены и обучены три глубокие полносвязные сети прямого распространения для задач бинарной классификации (IMDB), многоклассовой классификации (Reuters) и регрессии (Boston Housing); для каждой задачи рассмотрены особенности кодирования данных, выбора функции потерь и оценки качества, а результаты сопоставлены с базовыми моделями.

В задаче бинарной классификации IMDB сеть варианта 13 (16-32-16) достигла точности 0.8964 на тестовом подмножестве, незначительно превзойдя базовую модель (0.8946). Это показывает, что заданная вариантом конфигурация с функцией потерь MSE работает не хуже стандартного решения с бинарной перекрёстной энтропией.

В задаче многоклассовой классификации Reuters сеть варианта 13 (32-64-64) достигла точности 0.7467, тогда как базовая модель (64-64) показала несколько более высокий результат 0.7796. Разница объясняется заданным вариантом сочетанием функций активации и небольшим числом нейронов в первом скрытом слое, то есть неоптимальностью гиперпараметров варианта для данной задачи.

В задаче регрессии Boston Housing средняя абсолютная погрешность по четырём блокам перекрёстной проверки составила 2.94 тыс. долларов, что близко к результату базовой модели (2.77) и является приемлемой точностью для данного набора. Применение K-блочной кросс-валидации позволило получить более устойчивую оценку обобщающей способности на небольшой выборке. Все этапы работы выполнены, получены графики функции потерь и метрик качества, проведено сопоставление с базовыми классификаторами.